

Linux Filesystem Management - Study Notes

I. Filesystem Hierarchy Standard (FHS)

Root Directory Structure

```
$ ls -l /  
# Shows directories under root (/)
```

Key Directories

- **/** - Root directory of entire filesystem hierarchy
- **/bin** - Essential ready-to-run programs (binaries) like `ls`, `cp`
- **/boot** - Kernel boot loader files
- **/dev** - Device files
- **/etc** - Core system configuration files (no binaries)
- **/home** - Personal user directories
- **/lib** - Library files used by binaries
- **/media** - Mount point for removable media (USB drives)
- **/mnt** - Temporarily mounted filesystems
- **/opt** - Optional application software packages
- **/proc** - Information about currently running processes
- **/root** - Root user's home directory
- **/run** - Runtime information since last boot
- **/sbin** - Essential system binaries (usually root-only)
- **/srv** - Site-specific data served by system
- **/tmp** - Temporary files storage
- **/usr** - User-installed software and utilities
- **/var** - Variable data (logs, caches, user tracking)

2. Filesystem Types

Virtual File System (VFS)

- **Purpose:** Abstraction layer between applications and filesystem types
- **Function:** Allows applications to work with any filesystem type

Journaling

- **Purpose:** Maintains filesystem consistency
- **Process:**
 1. Writes operations to log file (journal) before execution
 2. Marks tasks as complete after execution
 3. Maintains consistent filesystem state
- **Benefits:**
 - Faster boot times (checks journal instead of entire filesystem)
 - Recovery from unexpected shutdowns
 - Prevents data corruption

Common Filesystem Types

ext4

- **Description:** Most current native Linux filesystem
- **Compatibility:** Works with ext2 and ext3
- **Capacity:** Up to 1 exabyte disk volumes, 16 terabyte files
- **Status:** Standard Linux filesystem choice

Btrfs

- **Full Name:** "Better or Butter FS"
- **Features:** Snapshots, incremental backups, performance improvements
- **Status:** Widely available but not fully stable

XFS

- **Description:** High-performance journaling filesystem
- **Best Use:** Systems with large files (media servers)

Other Types

- **NTFS/FAT:** Windows filesystems
- **HFS+:** Macintosh filesystem

Check Current Filesystems

```
pete@icebox:~$ df -T
```

Filesystem	Type	1K-blocks	Used	Available	Use%	Mounted on
------------	------	-----------	------	-----------	------	------------

/dev/sda1	ext4	6461592	2402708	3707604	40% /
udev	devtmpfs	501356	4	501352	1% /dev
tmpfs	tmpfs	102544	1068	101476	2% /run
/dev/sda6	xfs	13752320	460112	13292208	4% /home

3. Anatomy of a Disk

Basic Concepts

- **Partitions:** Subdivisions of hard disks creating multiple block devices
- **Examples:** `/dev/sda` (whole disk), `/dev/sda1` (first partition)
- **Purpose:** Separate data, use different filesystem types

Partition Table

- **Function:** Tells system how disk is partitioned
- **Information:** Start/end points, bootable partitions, sector allocation
- **Types:** MBR (Master Boot Record) and GPT (GUID Partition Table)

MBR (Master Boot Record)

- **Status:** Traditional standard
- **Partition Types:** Primary, Extended, Logical
- **Limitations:**
 - Maximum 4 primary partitions
 - Extended partition contains logical partitions
 - Only one extended partition per disk
 - Supports disks up to 2 terabytes

GPT (GUID Partition Table)

- **Status:** New standard
- **Advantages:**
 - Only one partition type
 - Many partitions possible
 - Each partition has globally unique ID (GUID)
 - Used with UEFI booting

Filesystem Structure Components

1. **Boot Block:** First few sectors, contains OS boot information
2. **Super Block:** Contains filesystem metadata (size, block information)

3. **Inode Table**: Database managing files and directories

4. **Data Blocks**: Actual file and directory data

Partition Table Examples

MBR Example

```
pete@icebox:~$ sudo parted -l
```

Model: Seagate (scsi)

Disk /dev/sda: 21.5GB

Sector size (logical/physical): 512B/512B

Partition Table: msdos

Number	Start	End	Size	Type	File system	Flags
1	1049kB	6860MB	6859MB	primary	ext4	boot
2	6861MB	21.5GB	14.6GB	extended		
5	6861MB	7380MB	519MB	logical	linux-swap(v1)	
6	7381MB	21.5GB	14.1GB	logical	xfs	

GPT Example

Model: Thumb Drive (scsi)

Disk /dev/sdb: 4041MB

Sector size (logical/physical): 512B/512B

Partition Table: gpt

Number	Start	End	Size	File system	Name	Flags
1	17.4kB	1000MB	1000MB		first	
2	1000MB	4040MB	3040MB		second	

4. Disk Partitioning

Partitioning Tools

- **fdisk**: Basic command-line tool (no GPT support)
- **parted**: Command-line tool (supports MBR and GPT)
- **gparted**: GUI version of parted
- **gdisk**: GPT-only version of fdisk

Using parted

Launch parted

```
sudo parted
```

Select Device

```
select /dev/sdb2
```

View Current Partition Table

```
(parted) print
```

Create Partition

```
mkpart primary 123 4567  
# Creates partition from sector 123 to 4567
```

Resize Partition

```
resize 2 1245 3456  
# Resizes partition 2 to new start/end points
```

5. Creating Filesystems

Basic Filesystem Creation

```
$ sudo mkfs -t ext4 /dev/sdb2  
# Creates ext4 filesystem on /dev/sdb2
```

Filesystem Creation Notes

- Only create filesystems on newly partitioned or repartitioned disks
- Creating filesystem on existing one will corrupt data
- Use `mkfs` (make filesystem) command

6. mount and umount

Basic Mounting

```
$ sudo mount -t ext4 /dev/sdb2 /mydrive
# -t: filesystem type
# /dev/sdb2: device location
# /mydrive: mount point
```

Prerequisites

1. Create mount point: `mkdir /mydrive`
2. Specify filesystem type, device location, and mount point

Unmounting

```
$ sudo umount /mydrive
# or
$ sudo umount /dev/sdb2
```

Using UUIDs

View UUIDs

```
pete@icebox:~$ sudo blkid

/dev/sda1: UUID="130b882f-7d79-436d-a096-1e594c92bb76" TYPE="ext4"
/dev/sda5: UUID="22c3d34b-467e-467c-b44d-f03803c2c526" TYPE="swap"
/dev/sda6: UUID="78d203a0-7c18-49bd-9e07-54f44cdb5726" TYPE="xfs"
```

Mount by UUID

```
sudo mount UUID=130b882f-7d79-436d-a096-1e594c92bb76 /mydrive
```

Benefits of UUIDs

- Device names can change between boots
- UUIDs remain constant
- Essential for automatic mounting

7. /etc/fstab

Purpose

- **File:** `/etc/fstab` (pronounced "eff es tab")
- **Function:** Permanent list of filesystems for automatic mounting at startup

Example fstab File

```
pete@icebox:~$ cat /etc/fstab

UUID=130b882f-7d79-436d-a096-1e594c92bb76 /          ext4
relatime,errors=remount-ro 0          1
UUID=78d203a0-7c18-49bd-9e07-54f44cdb5726 /home      xfs      relatime
0          2
UUID=22c3d34b-467e-467c-b44d-f03803c2c526 none       swap     sw
0          0
```

Field Structure

1. **UUID:** Device identifier
2. **Mount Point:** Directory where filesystem is mounted
3. **Filesystem Type:** ext4, xfs, swap, etc.
4. **Options:** Mount options (see man page)
5. **Dump:** Backup utility setting (usually 0)
6. **Pass:** fsck check order (0 = no check, 1 = first, 2 = second)

8. Swap

Swap Purpose

- **Function:** Virtual memory allocation
- **Process:** Swaps idle process memory to disk when RAM is low

Swap Partition Example

Number	Start	End	Size	Type	File system	Flags
5	6861MB	7380MB	519MB	logical	linux-swap(v1)	

Setting Up Swap

```
# 1. Ensure partition is empty
# 2. Initialize swap area
```

```
$ mkswap /dev/sdb2

# 3. Enable swap device
$ swapon /dev/sdb2

# 4. Add to /etc/fstab for persistence (use 'sw' as filesystem type)

# 5. Disable swap
$ swapoff /dev/sdb2
```

Swap Size Guidelines

- **Traditional:** Allocate 2x RAM size
- **Modern:** Systems usually have sufficient RAM

9. Disk Usage

df Command (Disk Free)

```
pete@icebox:~$ df -h
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda1	6.2G	2.3G	3.6G	40%	/

- **Purpose:** Shows mounted filesystem utilization
- **Flag:** `-h` for human-readable format

du Command (Disk Usage)

```
$ du -h
# Shows disk usage of current directory

$ du -h /
# Shows disk usage starting from root (can be cluttered)
```

Memory Aid

- **df:** Check how much disk space is **free**
- **du:** Check **disk usage** of files/directories

10. Filesystem Repair

fsck Command

```
sudo fsck /dev/sda
```

fsck Purpose

- **Function:** Check filesystem consistency and repair corruption
- **When Used:**
 - Automatically during boot (before mounting)
 - Manually when filesystem is severely corrupted

Important Notes

- Use only when filesystem is **unmounted**
- Best performed from rescue disk
- Runs automatically during normal boot process

II. Inodes

What is an Inode?

- **Definition:** Index node - entry in filesystem's database
- **Purpose:** Stores metadata about files (everything except filename and content)

Inode Contents

- File type (regular file, directory, device, etc.)
- Owner and group
- Access permissions
- Timestamps:
 - **mtime:** Last modification time
 - **ctime:** Last attribute change time
 - **atime:** Last access time
- Number of hard links
- File size
- Number of allocated blocks
- **Pointers to data blocks** (most important!)

Inode Creation and Allocation

- Created when filesystem is created

- Space allocated based on disk volume algorithms
- Can run out of inodes (prevents creating new files)

Check Inode Usage

```
$ df -i
# Shows inode usage statistics
```

View Inode Information

```
pete@icebox:~$ ls -li

140 drwxr-xr-x 2 pete pete 6 Jan 20 20:13 Desktop
141 drwxr-xr-x 2 pete pete 6 Jan 20 20:01 Documents
# First field shows inode number
```

Detailed Inode Information

```
pete@icebox:~$ stat ~/Desktop/

File: '/home/pete/Desktop/'
Size: 6          Blocks: 0          IO Block: 4096   directory
Device: 806h/2054d    Inode: 140         Links: 2
Access: (0755/drwxr-xr-x)  Uid: ( 1000/   pete)   Gid: ( 1000/   pete)
Access: 2016-01-20 20:13:50.647435982 -0800
Modify: 2016-01-20 20:13:06.191675843 -0800
Change: 2016-01-20 20:13:06.191675843 -0800
Birth: -
```

How Inodes Locate Files

- **Structure:** Each inode contains 15 pointers
- **Direct Pointers:** First 12 point directly to data blocks
- **Indirect Pointers:**
 - 13th: Points to block containing more pointers
 - 14th: Points to double-indirect block
 - 15th: Points to triple-indirect block
- **Purpose:** Uniform inode structure for files of different sizes

12. Symbolic Links (Symlinks) and Hard Links

Link Count

- **Location:** Third field in `ls -l` output
- **Definition:** Total number of hard links to a file

Symbolic Links (Symlinks)

- **Equivalent:** Linux version of Windows shortcuts
- **Function:** Link to another file by filename
- **Characteristics:**
 - Use different inode numbers
 - Can span different filesystems
 - Break if original file is deleted

Creating Symlinks

```
pete@icebox:~/Desktop$ echo 'myfile' > myfile
pete@icebox:~/Desktop$ ln -s myfile myfilelink
pete@icebox:~/Desktop$ ls -li

total 12
 151 -rw-rw-r-- 1 pete pete 7 Jan 21 21:36 myfile
93403 lrwxrwxrwx 1 pete pete 6 Jan 21 21:39 myfilelink -> myfile
```

Symlink Command

```
$ ln -s myfile mylink
# -s: symbolic link flag
# myfile: target file
# mylink: link name
```

Hard Links

- **Function:** Another file with link to same inode
- **Characteristics:**
 - Same inode number as original
 - Cannot span filesystems
 - Original file persists until all hard links deleted

Creating Hard Links

```
pete@icebox:~/Desktop$ ln myfile2 myhardlink
pete@icebox:~/Desktop$ ls -li

total 16
 151 -rw-rw-r-- 1 pete pete 7 Jan 21 21:36 myfile
93401 -rw-rw-r-- 2 pete pete 8 Jan 21 21:36 myfile2
93401 -rw-rw-r-- 2 pete pete 8 Jan 21 21:36 myhardlink
# Notice same inode number (93401) and link count (2)
```

Hard Link Command

```
$ ln somefile somelink
# No -s flag for hard links
```

Key Differences

Feature	Symbolic Links	Hard Links
Inode	Different	Same
Filesystem	Can span	Cannot span
Target deletion	Link breaks	Link survives
Link count	Not affected	Increases